

Site Copilot

Agentic RFI and Daily Report assistant for construction jobsites.

Arsen Khanguieldyan

smartpneu.contact@gmail.com

Portfolio submission for Suffolk Construction — Site AI Engineer (Boston, MA)

TRY IT LIVE

Live demo suffolk-construction.up.railway.app

Source code github.com/smartpneucontact-sketch/Sufflk

Model Anthropic Claude Sonnet 4.6 · ~\$0.05 per RFI triage

SUMMARY

Site Copilot is a working portfolio demo aligned to the Suffolk Site AI Engineer role. Two production-shaped AI agents — RFI triage and Daily Construction Report drafting — run on Claude Sonnet 4.6 with Agentic RAG over a synthetic project corpus. The stack uses every component named in the JD: FastAPI, hybrid retrieval mapped 1:1 to OpenSearch, JSONL telemetry mapped to Databricks bronze, and a Terraform sketch for an AWS Bedrock deployment.

THE PROBLEM

On a typical Suffolk-scale project, project engineers field hundreds of RFIs. Industry baseline for response is ~9.7 days. End-of-shift, a superintendent spends 30–60 minutes translating sloppy field notes into a structured Daily Construction Report. Both are high-frequency, citation-heavy, and bound by the project's own documents — the textbook shape of an Agentic RAG workflow. Suffolk's own JD names both use cases by name.

WHAT IT SHIPS

AGENT 01

RFI Triage

Reads an incoming RFI, retrieves relevant spec sections and similar prior RFIs from the project corpus, drafts a citation-backed response, classifies urgency, estimates rough cost and schedule impact (multiple scenarios), and flags whether escalation to the Structural Engineer of Record is required.

Why it matters: compresses ~9.7-day industry baseline to ~30 seconds and ~\$0.05 per RFI. Cites every claim. Refuses to over-escalate when the spec is unambiguous and prior EOR rulings are on record.

AGENT 02

Daily Construction Report

Takes raw, fragmented end-of-shift superintendent notes and produces a structured DCR with normalized crews, areas worked, equipment, deliveries, follow-ups, and an executive summary. Risks are bucketed into a fixed taxonomy (safety, schedule, quality, weather, material, coordination) so downstream BI can aggregate.

Why it matters: turns 30–60 minutes of nightly paperwork into a 30-second review. Open RFIs in the notes are

auto-extracted; weather forecasts are flagged against pending work.

HOW IT MAPS TO SUFFOLK'S NAMED STACK

Every component in this demo has a 1:1 production target. The swap from local demo to deployed AWS is mechanical, not architectural — the LLMClient interface mirrors Bedrock's Converse API, the VectorStore Protocol mirrors OpenSearch's kNN, and the trace JSONL format ingests directly into Databricks bronze with no transformation.

THIS DEMO	SUFFOLK PRODUCTION
Anthropic SDK + Claude Sonnet 4.6	AWS Bedrock — anthropic.claude-sonnet-4-6
BM25 (+ optional Chroma), fused via RRF	OpenSearch hybrid (BM25 + kNN)
FastAPI + Pydantic	FastAPI + Pydantic
JSONL traces on disk	CloudWatch Logs → Databricks bronze
Streamlit / HTML single-page UI	Teams / SharePoint apps
GitHub Actions CI (lint + tests + eval gate)	Same
infra/terraform/ sketch (ECS Fargate)	Production Terraform / CloudFormation
Tool-use loop in agents/base.py	Bedrock Converse multi-tool loop

LLMOPS STORY

The JD calls out 'monitor latency, cost, adoption, and drift'. The demo ships all four:

- Latency: every LLM call and tool call is wrapped in a span with elapsed_ms. p50 / p95 per agent is a one-line awk on the JSONL.
- Cost: per-call cost_usd computed from per-million-token pricing in llm.py. Cost per RFI is ~\$0.05 at current Sonnet 4.6 pricing — at 50 RFIs/site/week, Anthropic bill is under \$15/site/month.
- Adoption: every run carries actor_id + source_surface tags; weekly active users and auto-resolution rate are one query away.
- Drift: rolling window watches response length, citation count, and escalation rate for >3-sigma moves. Production target is Bedrock Evaluation or Databricks ML monitoring.
- Eval gate: 4 RFI cases with expected behaviors in evals/rfi_cases.yaml. CI runs them in mock-LLM mode on every PR; merges block on scorer failures.

WHY THIS FITS THE SITE AI ENGINEER ROLE

The JD describes a 'walk-the-jobsite, ship MVP agents in days' role partnered with project AI Champions. Site Copilot is exactly the artifact that role produces: pick a workflow pain, build the agent against the project's own corpus, ship it on the central AI Studio's stack, stand up the LLMOps spine, hand off cleanly. The JD also lists 'Hands-on expertise with Claude Code' as required — this entire project was built in Claude Code with Claude as the runtime, which doubles as a credibility signal.

WHAT IS INTENTIONALLY NOT BUILT

This is a one-week portfolio demo, not a production deploy. The README's gap list is direct:

- Corpus is synthetic but realistic — real specs would need OCR, table extraction, and drawing-callout linking.
- Terraform deploys infra but no autoscaling, alarms, or multi-account boundaries — those are stubbed in the README, not the .tf files.

- No Procore or OpenSpace integration yet — those are the natural next two integrations (Procore for RFI/DCR submission, OpenSpace for photo metadata feeding the DCR risk surfacing).
- Drift detector is the smallest thing that could plausibly trigger an alert; production needs proper distribution tests (KS, PSI) against a labeled baseline window.

IN ONE LINE

Working agentic AI on Suffolk's exact stack vocabulary, against Suffolk's exact named use cases, shippable to a real jobsite in a sprint.